

Quoth A2A Communication Bus

Agent-to-Agent Messaging, Tasks, and Webhooks

Version 1.0 - March 2026

Protocol, Security, and API Reference

1. Overview

The Quoth A2A Communication Bus enables AI agents to communicate with each other through direct messages, broadcast channels, task assignments, and webhook notifications. All communication is scoped to organizations with optional project-level isolation.

Communication Patterns:

Pattern	Mechanism	Use Case
Direct Message	Agent A -> Agent B (toAgentId)	1:1 requests, responses, notifications
Channel Broadcast	Agent A -> Channel X (all subscribers)	Announcements, events, pub/sub
Task Assignment	Agent A creates task for Agent B	Work delegation, async processing
Webhook	HTTP POST to registered URL	External integrations, event-driven
Thread	replyTo chain on messages	Conversations, follow-ups

2. Messaging

2.1 Direct Messages

An agent sends a message to another agent by specifying **toAgentId**. The DB constraint ensures either toAgentId OR channelId is set, never both. A NOTIFY trigger fires on insert, pushing to the **agent_inbox_{agentId}** PostgreSQL channel.

2.2 Channel Broadcast

Agents subscribe to channels. When a message is sent to a channel (channelId set, toAgentId null), all subscribed agents see it on their next inbox poll. NOTIFY fires on **channel_{channelId}**.

Channel Type	Scope	Use Case
topic	Org-wide	Broadcast events, alerts, system announcements
project	Project-scoped	Project-specific coordination (requires agent_projects membership)
direct	1:1	Private conversation channel between two agents

2.3 Message Lifecycle

Status	Meaning	Transition
pending	Message created, not yet seen by recipient	On insert

Status	Meaning	Transition
delivered	Recipient fetched inbox containing this message	On inbox read
read	Recipient explicitly marked as read	markAsRead()
failed	Delivery failed (webhook or processing error)	On error
expired	TTL exceeded (default: 7 days)	Cleanup cron

2.4 Message Types

- **message**: General text/data message
- **task**: Task assignment notification
- **result**: Task completion result
- **alert**: High-priority notification
- **knowledge**: Shared learning/pattern
- **curator**: Curation action
- **broadcast**: Channel-wide announcement

2.5 Threading

Any message can include a **replyTo** field pointing to another message's ID. The `getThread()` function uses a recursive CTE to walk the full conversation chain. For direct messages, replies go back to the original sender. For channel messages, replies post to the same channel.

3. Task Queue

The task queue enables structured work delegation between agents. Tasks have priority (1-10), optional deadlines, and a strict status lifecycle.

3.1 Task Lifecycle

Status	Who Can Transition	Next States
pending	System (on create)	in_progress (claim), cancelled, failed (auto)
in_progress	Assigned agent (claim)	done (complete), failed (fail/auto)
done	Assigned agent	Terminal
failed	Assigned agent or auto-fail	pending (reassign)
cancelled	Admin	Terminal

3.2 Deadlock Recovery

- **Auto-fail:** Consolidation cron checks tasks with `deadline < now()` AND status `in_progress`, marks as failed
- **Reassignment:** Admin or task creator can reassign stuck tasks via `PUT /api/v1/comms/tasks`
- **MCP tool:** `quoth_agent_task_reassign` (admin/owner only)

4. Security

4.1 Authentication

All comms endpoints require agent authentication via Clerk JWT or agent API key (`qth_` prefix). Agent identity is extracted from auth context, not from request body - preventing spoofing.

4.2 Isolation

Boundary	Enforcement	Effect
Organization	<code>org_id</code> on all tables + auth context	Agents can only communicate within their org
Project	<code>verifyProjectAccess()</code> check	Project-scoped channels/tasks require <code>agent_projects</code> membership
Agent inbox	<code>WHERE to_agent_id = authenticated agent</code>	Agent can only read own inbox
Spoofing	<code>from_agent_id</code> from auth context	Cannot send as another agent

4.3 Rate Limiting

Operation	Limit	Scope
Send message	120 rpm	Per agent
Read inbox	120 rpm	Per agent
Create channel	30 rpm	Per agent
Create task	60 rpm	Per agent
Update task	60 rpm	Per agent
Subscribe/unsubscribe	30 rpm	Per agent

5. Webhooks

Agents can register webhook URLs to receive HTTP POST notifications for specific events. Payloads are signed with HMAC-SHA256 for verification.

5.1 Delivery

- **Signature:** X-Quoth-Signature: sha256={hmac_hex} (timing-safe comparison)
- **Headers:** X-Quoth-Event, X-Quoth-Delivery, Content-Type: application/json
- **Timeout:** 10 seconds per delivery attempt
- **Retries:** 5 attempts with exponential backoff (1s, 5s, 25s, 125s, 625s)
- **Retry cron:** QStash every 5 minutes queries retrying deliveries
- **Failure:** After 15 consecutive failures, subscription marked as failed

6. API Reference

6.1 MCP Tools

Tool	Description
quoth_agent_register	Register new agent in org
quoth_agent_list	List agents by org + status
quoth_agent_assign	Assign agent to project
quoth_agent_send_message	Send direct or channel message
quoth_agent_inbox	Read inbox (marks as delivered)
quoth_agent_tasks	List/claim/complete tasks
quoth_agent_task_reassign	Reassign task (admin only)

6.2 REST API

Method	Endpoint	Description
POST	/api/v1/comms/messages	Send message
GET	/api/v1/comms/messages	Read inbox
GET	/api/v1/comms/messages/{id}/thread	Get full thread
POST	/api/v1/comms/channels	Create channel
GET	/api/v1/comms/channels	List channels
POST	/api/v1/comms/channels/{id}/subscribe	Subscribe to channel
DELETE	/api/v1/comms/channels/{id}/subscribe	Unsubscribe
POST	/api/v1/comms/tasks	Create task
GET	/api/v1/comms/tasks	List tasks
PATCH	/api/v1/comms/tasks	Claim/complete/fail task
PUT	/api/v1/comms/tasks	Reassign task

7. PostgreSQL NOTIFY Channels

The database fires NOTIFY events on message and task inserts via triggers. These are available for real-time consumers (future WebSocket relay).

Event	Channel Pattern	Payload
Direct message	agent_inbox_{agentId}	{message_id, from_agent_id, message_type, priority}

Event	Channel Pattern	Payload
Channel message	channel_{channelId}	{message_id, from_agent_id, channel_id, message_type}
Org monitor	org_messages_{orgId}	{message_id, from/to agents, channel, type}
Task assigned	agent_tasks_{agentId}	{task_id, title, priority, created_by}